

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\integration\
TrapezoidIntegrator.java

- Specification

module TrapezoidIntegrator

type

```
relativeAccuracy: real
absoluteAccuracy: real
minimalIterationCount: int
maximalIterationCount: int
min: real
max: real
maxEval: int
evaluations_used: int
iterations_used: int
converged_by_value_accuracy: bool
converged_by_interval_shrinkage: bool
not_converged_by_accuracy: bool
ex: set of {
    None,
    NullArgumentException,
    NotStrictlyPositiveException,
    NumberIsTooSmallException,
    NumberIsTooLargeException,
    NotANumberException,
    NotFiniteNumberException,
    TooManyEvaluationsException,
    MaxCountExceededException
}
```

process integrate(f, min, max, maxEval,
relativeAccuracy, absoluteAccuracy,
minimalIterationCount, maximalIterationCount: real)
result: real, ex: set of { ... }

Functional Scenario Form

```
f = 0                                && ex = NullArgumentException
||
maxEval <= 0                          && ex =
NotStrictlyPositiveException ||
relativeAccuracy <= 0    && ex = NotStrictlyPositiveException ||
minimalIterationCount < MIN_TRAPEZOID_ITER    && ex =
NumberIsTooSmallException ||
```

```

    maximalIterationCount > MAX_TRAPEZOID_ITER          && ex =
NumberIsTooLargeException ||
    minimalIterationCount >= maximalIterationCount      && ex =
NumberIsTooSmallException ||
    isNaN=1          && ex = NotANumberException ||
    isInfinite=1     && ex = NotFiniteNumberException ||
    min >= max          && ex =
NumberIsTooLargeException ||
    evaluations_used > maxEval          && ex = TooManyEvaluationsException ||
    iterations_used >= maximalIterationCount          && ex =
MaxCountExceededException ||
    evaluations_used          <=          maxEval
    && ex = None

```

end_process

end_module

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;

```

```

import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeException;
import org.apache.commons.math4.legacy.exception.NotANumberException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberException;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsException;
import org.apache.commons.math4.legacy.exception.MaxCountExceededException;
/**
 * Implements the <a href="http://mathworld.wolfram.com/TrapezoidalRule.html">
 * Trapezoid Rule</a> for integration of real univariate functions.
 *
 * See <b>Introduction to Numerical Analysis</b>, ISBN 038795452X, chapter 3.
 *
 * <p>
 * The function should be integrable.
 *
 * <p>
 * <em>Caveat:</em> At each iteration, the algorithm refines the estimation by
 * evaluating the function twice as many times as in the previous iteration;
 * When specifying a {@link #integrate(int,UnivariateFunction,double,double)
 * maximum number of function evaluations}, the caller must ensure that it
 * is compatible with the {@link #TrapezoidIntegrator(int,int) requested
 * minimal number of iterations}.
 *
 * @since 1.2
 */
public class TrapezoidIntegrator extends BaseAbstractUnivariateIntegrator {
    /** Maximum number of iterations for trapezoid. */
    private static final int TRAPEZOID_MAX_ITERATIONS_COUNT = 30;
    /** Intermediate result. */

```

```

    private double s;
    /**
     * Build a trapezoid integrator with given accuracies and iteration
s counts.
     * @param relativeAccuracy relative accuracy of the result
     * @param absoluteAccuracy absolute accuracy of the result
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations
     * @throws org.apache.commons.math4.legacy.exception.NotStrictlyPos
itiveException
     * if {@code minimalIterationCount <= 0}.
     * @throws org.apache.commons.math4.legacy.exception.NumberIsTooSma
llException
     * if {@code maximalIterationCount < minimalIterationCount}.
     * is lesser than or equal to the minimal number of iterations
     * @throws NumberIsTooLargeException if {@code maximalIterationCoun
t > 30}.
    */
    public TrapezoidIntegrator(final double relativeAccuracy,
                               final double absoluteAccuracy,
                               final int minimalIterationCount,
                               final int maximalIterationCount) {
        super(relativeAccuracy, absoluteAccuracy, minimalIterationCount
, maximalIterationCount);
        if (maximalIterationCount > TRAPEZOID_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
                                                TRAPEZOID_MAX_ITERATION
S_COUNT, false);
        }
    }
    /**
     * Build a trapezoid integrator with given iteration counts.
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations
     * @throws org.apache.commons.math4.legacy.exception.NotStrictlyPos
itiveException
     * if {@code minimalIterationCount <= 0}.
     * @throws org.apache.commons.math4.legacy.exception.NumberIsTooSma
llException
     * if {@code maximalIterationCount < minimalIterationCount}.
     * is lesser than or equal to the minimal number of iterations
     * @throws NumberIsTooLargeException if {@code maximalIterationCoun
t > 30}.
    */

```

```

    public TrapezoidIntegrator(final int minimalIterationCount,
                               final int maximalIterationCount) {
        super(minimalIterationCount, maximalIterationCount);
        if (maximalIterationCount > TRAPEZOID_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount,
                                                  TRAPEZOID_MAX_ITERATION
S_COUNT, false);
        }
    }

    /**
     * Construct a trapezoid integrator with default settings.
     */
    public TrapezoidIntegrator() {
        super(DEFAULT_MIN_ITERATIONS_COUNT, TRAPEZOID_MAX_ITERATIONS_CO
UNT);
    }

    /**
     * Compute the n-th stage integral of trapezoid rule. This function
     * should only be called by API <code>integrate()</code> in the pac
kage.
     * To save time it does not verify arguments - caller does.
     * <p>
     * The interval is divided equally into  $2^n$  sections rather than an
     * arbitrary m sections because this configuration can best utilize
the
     * already computed values.</p>
     *
     * @param baseIntegrator integrator holding integration parameters
     * @param n the stage of 1/2 refinement, n = 0 is no refinement
     * @return the value of n-th stage integral
     * @throws org.apache.commons.math4.legacy.exception.TooManyEvaluat
ionsException if the maximal number of evaluations
     * is exceeded.
     */
    double stage(final BaseAbstractUnivariateIntegrator baseIntegrator,
final int n) {
        if (n == 0) {
            final double max = baseIntegrator.getMax();
            final double min = baseIntegrator.getMin();
            s = 0.5 * (max - min) *
                (baseIntegrator.computeObjectiveValue(min) +
                 baseIntegrator.computeObjectiveValue(max));
            return s;
        } else {

```

```

        final long np = 1L << (n-
1);        // number of new points in this stage
        double sum = 0;
        final double max = baseIntegrator.getMax();
        final double min = baseIntegrator.getMin();
        // spacing between adjacent new points
        final double spacing = (max - min) / np;
        double x = min + 0.5 * spacing;    // the first new point
        for (long i = 0; i < np; i++) {
            sum += baseIntegrator.computeObjectiveValue(x);
            x += spacing;
        }
        // add the new sum to previously calculated result
        s = 0.5 * (s + sum * spacing);
        return s;
    }
}
/** {@inheritDoc} */
@Override
protected double doIntegrate() {
    double oldt = stage(this, 0);
    iterations.increment();
    while (true) {
        final int i = iterations.getCount();
        final double t = stage(this, i);
        if (i >= getMinimalIterationCount()) {
            final double delta = JdkMath.abs(t - oldt);
            final double rLimit =
                getRelativeAccuracy() * (JdkMath.abs(oldt) + JdkMat
h.abs(t)) * 0.5;
            if (delta <= rLimit || delta <= getAbsoluteAccuracy())
{
                return t;
            }
        }
        oldt = t;
        iterations.increment();
    }
}
}
class TIVT {
    // ---- helpers ----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
}

```

```

    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ---- functions ----
    static final UnivariateFunction F_ZERO = x -> 0.0; // integral = 0
    static final UnivariateFunction F_CONST = x -> 5.0; // ∫ 5 dx = 5(b-a)
    static final UnivariateFunction F_LIN = x -> x - 2.0; // ∫_0^4 (x-2) dx = 0
    static final UnivariateFunction F_SIN = Math::sin; // ∫_0^π sin = 2
    public void Vtest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 2;
        final int MAX_ITER = 16;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));
        TrapezoidIntegrator T = new TrapezoidIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
        // VT01: f == null -> NullPointerException
        expectThrow("VT01 f=null",
            () -> T.integrate(1000, null, 0.0, 1.0),
            NullPointerException.class);
        // VT02: maxEval <= 0 -> NotStrictlyPositiveException
        expectThrow("VT02 maxEval=0",
            () -> T.integrate(0, F_SIN, 0.0, 1.0),
            NotStrictlyPositiveException.class);
        // VT03: rel<=0 AND abs<=0 (ctor) -> NotStrictlyPositiveException
        expectThrow("VT03 rel<=0 AND abs<=0 (ctor)",

```

```

        () -> new TrapezoidIntegrator(0.0, 0.0, 2, 10),
        NotStrictlyPositiveException.class);
// VT04: minIter < 1 (ctor) -> NumberIsTooSmallException
expectThrow("VT04 minIter<1 (ctor)",
        () -> new TrapezoidIntegrator(1e-8, 1e-8, 0, 10),
        NumberIsTooSmallException.class);
// VT05: maxIter > 64 (ctor) -> NumberIsTooLargeException
expectThrow("VT05 maxIter>64 (ctor)",
        () -> new TrapezoidIntegrator(1e-8, 1e-8, 1, 128),
        NumberIsTooLargeException.class);
// VT06: minIter == maxIter (ctor) -> NumberIsTooSmallException
expectThrow("VT06 minIter==maxIter (ctor)",
        () -> new TrapezoidIntegrator(1e-8, 1e-8, 5, 5),
        NumberIsTooSmallException.class);
// VT07: NaN endpoints -> NotANumberException
expectThrow("VT07 NaN at min",
        () -> T.integrate(1000, F_LIN, Double.NaN, 1.0),
        NotANumberException.class);
// VT08: Infinite endpoints -> NotFiniteNumberException
expectThrow("VT08 -Infinity at min",
        () -> T.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY
, 1.0),
        NotFiniteNumberException.class);
// VT09: min ≥ max (strict) -> NumberIsTooLargeException
expectThrow("VT09 min==max",
        () -> T.integrate(1000, F_CONST, 3.14, 3.14),
        NumberIsTooLargeException.class);
// VT10: TooManyEvaluationsException
expectThrow("VT10 budget exhausted (maxEval=1)",
        () -> T.integrate(1, F_SIN, 0.0, 10.0),
        TooManyEvaluationsException.class);
// VT11:
try {
    TrapezoidIntegrator T_tight = new TrapezoidIntegrator(1e-
30, 1e-30, 1, 2);
    expectThrow("VT11 hit max iterations (tight accuracy, small
band)",
        () -> T_tight.integrate(1 << 26, F_SIN, 0.0, 100.0)
,
        MaxCountExceededException.class);
} catch (Throwable t) {
    System.out.println("VT11 note: " + t.getClass().getSimpleName()
+ " - " + t.getMessage());
}

```

```

// VT12:
headline("VT12 ZERO small interval within budget");
double r12 = T.integrate(BIG_BUDGET, F_ZERO, -1e-8, 1e-8);
System.out.printf("VT12 result=%.17g%n", r12);
// VT13
headline("VT13 ctor(minIter,maxIter) mid-
range success (2,10) -> else-path");
TrapezoidIntegrator T_mOnly_ok2 = new TrapezoidIntegrator(2, 10
); // 10 << TRAPEZOID_MAX(64)
double vt23 = T_mOnly_ok2.integrate(1 << 16, F_CONST, 0.0, 1e-
3);
System.out.printf("VT13 result=%.17g (expected=%g)%n", vt23, 5.
0 * 1e-3);
// VT14
headline("VT14 near-degenerate but valid");
double a16 = 3.0;
double eps16 = Math.max(1e-9, 16 * Math.ulp(a16));
double r16 = T.integrate(BIG_BUDGET, F_CONST, a16, a16 + eps16)
;
System.out.printf("VT14 result=%.17g (expected=%g, eps=%g)%n",
r16, 5.0 * eps16, eps16);
// VT15
headline("VT15 tight accuracy on tiny interval");
TrapezoidIntegrator Ttiny = new TrapezoidIntegrator(1e-14, 1e-
14, 2, 16);
double r18 = Ttiny.integrate(BIG_BUDGET, F_SIN, 0.0, 1e-6);
System.out.printf("VT15 result=%.17g%n", r18);
// VT16
headline("VT16 ctor(minIter,maxIter) boundary success (1,64)");
TrapezoidIntegrator T_mOnly_ok = new TrapezoidIntegrator(1, 64)
;
double vt21 = T_mOnly_ok.integrate(1 << 22, F_CONST, 0.0, 1e-
4);
System.out.printf("VT16 result=%.17g (expected=%g)%n", vt21, 5.
0 * 1e-4);
System.out.println("\n[Done] TrapezoidIntegrator V-
method cases executed.");
}
}
//ft
class TIFT {
// ---- helpers ----
private static void headline(String name) {
System.out.println("\n==== " + name + " =====");
}
}

```

```

    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ---- functions ----
    static final UnivariateFunction F_ZERO = x -> 0.0; // integral = 0
    static final UnivariateFunction F_CONST = x -> 5.0; // ∫ 5 dx = 5(b-a)
    static final UnivariateFunction F_LIN = x -> x - 2.0; // ∫_0^4 (x-2) dx = 0
    static final UnivariateFunction F_SIN = Math::sin; // ∫_0^π sin = 2
    public void Ftest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 2;
        final int MAX_ITER = 16;
        final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 12));
        TrapezoidIntegrator T = new TrapezoidIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
        // FT01: reversed interval with huge magnitudes -> NumberIsTooLargeException
        expectThrow("FT01 reversed extreme interval",
            () -> T.integrate(1000, F_SIN,
                5.005341669616643E307, -1.1209145429141500E308),
            NumberIsTooLargeException.class);
        // FT02: degenerate interval at extreme negative -> NumberIsTooLargeException
        expectThrow("FT02 degenerate interval (min==max, extreme)",
            () -> T.integrate(1000, F_CONST,

```

```

        -1.7976931348623157E308, -
1.7976931348623157E308),
        NumberIsTooLargeException.class);
// FT03: NaN at min -> NotANumberException
expectThrow("FT03 NaN min",
        () -> T.integrate(1000, F_LIN, Double.NaN, 1.0),
        NotANumberException.class);
// FT04: NaN at max -> NotANumberException
expectThrow("FT04 NaN max",
        () -> T.integrate(1000, F_SIN, 0.0, Double.NaN),
        NotANumberException.class);
// FT05: +Infinity at max -> NotFiniteNumberException
expectThrow("FT05 +Infinity at max",
        () -> T.integrate(1000, F_SIN, -
10.0, Double.POSITIVE_INFINITY),
        NotFiniteNumberException.class);
// FT06: -Infinity at min -> NotFiniteNumberException
expectThrow("FT06 -Infinity at min",
        () -> T.integrate(1000, F_CONST, Double.NEGATIVE_INFINITY, 10.0),
        NotFiniteNumberException.class);
// FT07: tiny budget on very wide interval -> TooManyEvaluation
sException
expectThrow("FT07 budget=1 on wide interval",
        () -> T.integrate(1, F_SIN, -1.0e3, 1.0e3),
        TooManyEvaluationsException.class);
// FT08: budget=1 on [0,  $\pi$ ] -> TooManyEvaluationsException
expectThrow("FT08 budget=1 on [0, $\pi$ ]",
        () -> T.integrate(1, F_SIN, 0.0, Math.PI),
        TooManyEvaluationsException.class);
// FT09: ZERO on tiny symmetric interval (success; integral=0)
headline("FT09 ZERO tiny symmetric interval");
double ft09 = T.integrate(BIG_BUDGET, F_ZERO, -1e-12, 1e-12);
System.out.printf("FT09 result=%.17g\n", ft09);
// FT10: CONST near-degenerate safe epsilon (success)
headline("FT10 CONST near-degenerate safe");
double a10 = 3.141592653589793;
double eps10 = Math.max(1e-9, 16 * Math.ulp(a10));
double ft10 = T.integrate(BIG_BUDGET, F_CONST, a10, a10 + eps10
);
System.out.printf("FT10 result=%.17g (expected=%g, eps=%g)\n",
ft10, 5.0 * eps10, eps10);
// FT11: SIN on random moderate interval (success; boosted budg
et)

```

```

        headline("FT11 SIN moderate random window");
        int bud11 = Math.max(BIG_BUDGET, 1 << 22);
        double ft11 = T.integrate(bud11, F_SIN, 1.23456789012345, 23.45
67890123456);
        System.out.printf("FT11 result=%.17g (budget=%d)%n", ft11, bud1
1);
        // FT12: LIN on wide finite random range (success; boosted budg
et)
        headline("FT12 LIN wide finite random");
        int bud12 = Math.max(BIG_BUDGET, 1 << 22);
        double ft12 = T.integrate(bud12, F_LIN, -
5.43210987654321E2, 7.65432109876543E2);
        System.out.printf("FT12 result=%.17g (budget=%d)%n", ft12, bud1
2);
        // FT13: f = null at call site -> NullArgumentException
        expectThrow("FT13 f=null at integrate",
            () -> T.integrate(1000, null, -2.0, 2.0),
            NullArgumentException.class);
        // FT14: tight accuracy + small iter band -> Likely MaxCountExc
eededException
        try {
            TrapezoidIntegrator T_tight_FZ = new TrapezoidIntegrator(1e
-30, 1e-30, 1, 2);
            expectThrow("FT14 hit max iterations (tight acc, small band
)",
                () -> T_tight_FZ.integrate(Math.max(BIG_BUDGET, 1 <
< 24), F_SIN, 0.0, 100.0),
                MaxCountExceededException.class);
        } catch (Throwable t) {
            System.out.println("FT14 note: " + t.getClass().getSimpleNa
me() + " - " + t.getMessage());
        }
        // FT15: ZERO over very wide finite (success; integral=0)
        headline("FT15 ZERO very wide finite");
        double ft15 = T.integrate(Math.max(BIG_BUDGET, 1 << 23),
            F_ZERO, -7.654321098765432E307, 7.654321098765432E307);
        System.out.printf("FT15 result=%.17g%n", ft15);
        // FT16: CONST huge-offset small window (success)
        headline("FT16 CONST huge-offset small window");
        double A16 = 1.2345678901234567E307;
        double ft16 = T.integrate(Math.max(BIG_BUDGET, 1 << 22), F_CONS
T, A16, A16 + 1.0);
        System.out.printf("FT16 result=%.17g (expected=%g)%n", ft16, 5.
0);

```

```

        System.out.println("\n[Done] TrapezoidIntegrator V-
method cases executed.");
    }
}
//Z3
class TIZ3 {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n=== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> e
xpected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expe
cted.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " -
" + t.getMessage());
            System.out.printf("[%s] %s threw expected\n",
                t.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
    // ----- simple functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0;
    static final UnivariateFunction F_SIN = Math::sin;
    static final UnivariateFunction F_LIN = x -> x - 2.0;
    public void Z3test() {
        final TrapezoidIntegrator T_OK = new TrapezoidIntegrator(1e-
3, 1e-3, 1, 2);
        // Z301 -----
        expectThrow("Z301 f=null",
            () -> T_OK.integrate(10, null, 0.0, 1.0),
            NullPointerException.class);
        // Z302 -----
        TrapezoidIntegrator T302 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
        expectThrow("Z302 maxEval<=0",
            () -> T302.integrate(0, F_ZERO, 0.0, 1.0),

```

```

        NotStrictlyPositiveException.class);
// Z303 -----
-----
expectThrow("Z303 rel<=0 AND abs<=0 (ctor)",
    () -> new TrapezoidIntegrator(0.0, 0.0, 1, 2),
    NotStrictlyPositiveException.class);
// Z304 -----
-----
expectThrow("Z304 minIter<1 (ctor)",
    () -> new TrapezoidIntegrator(0.5, 0.0, 0, 1),
    NumberIsTooSmallException.class);
// Z305 -----
-----
expectThrow("Z305 maxIter>64 (ctor)",
    () -> new TrapezoidIntegrator(0.5, 0.0, 1, 65),
    NumberIsTooLargeException.class);
// Z306 -----
-----
expectThrow("Z306 minIter>=maxIter (ctor)",
    () -> new TrapezoidIntegrator(0.0, 0.5, 1, 0),
    NumberIsTooSmallException.class);
// Z307 -----
-----
TrapezoidIntegrator T307 = new TrapezoidIntegrator(0.5, 0.0, 1,
2);
expectThrow("Z307 NaN at min",
    () -> T307.integrate(10, F_LIN, Double.NaN, 1.0),
    NotANumberException.class);
// Z308 -----
-----
TrapezoidIntegrator T308 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
expectThrow("Z308 NaN at max",
    () -> T308.integrate(10, F_LIN, 0.0, Double.NaN),
    NotANumberException.class);
// Z309 -----
-----
TrapezoidIntegrator T309 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
expectThrow("Z309 -Infinity(min)",
    () -> T309.integrate(10, F_SIN, Double.NEGATIVE_INFINITY,
1.0),
    NotFiniteNumberException.class);

```

```

// Z310 -----
-----
TrapezoidIntegrator T310 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
expectThrow("Z310 +Infinity(max)",
    () -> T310.integrate(10, F_SIN, 0.0, Double.POSITIVE_IN
FINITY),
    NotFiniteNumberException.class);
// Z311 -----
-----
TrapezoidIntegrator T311 = new TrapezoidIntegrator(0.5, 0.0, 1,
2);
expectThrow("Z311 min==max",
    () -> T311.integrate(10, F_CONST, 0.0, 0.0),
    NumberIsTooLargeException.class);
// Z312 -----
-----
TrapezoidIntegrator T312 = new TrapezoidIntegrator(0.5, 0.0, 1,
2);
expectThrow("Z312 budget exhausted (maxEval=1 surrogate)",
    () -> T312.integrate(1, F_SIN, 0.0, 1.0),
    TooManyEvaluationsException.class);
// Z313 -----
-----
try {
    TrapezoidIntegrator T313 = new TrapezoidIntegrator(1e-
30, 1e-30, 1, 2);
    expectThrow("Z313 hit max iterations (tight acc, small band
)",
        () -> T313.integrate(1 << 24, F_SIN, 0.0, 100.0),
        MaxCountExceededException.class);
} catch (Throwable t) {
    System.out.println("Z313 note: " + t.getClass().getSimpleNa
me() + " - " + t.getMessage());
}
// Z314 -----
-----
headline("Z314 success (value accuracy surrogate)");
TrapezoidIntegrator T314 = new TrapezoidIntegrator(0.5, 0.0, 1,
2);
double z314 = T314.integrate(1000, F_CONST, 0.0, 1e-4);
System.out.printf("Z314 result=%.17g (expected≈%g)%n", z314, 5.
0 * 1e-4);

```

```

// Z315 -----
headline("Z315 success (interval shrinkage surrogate)");
TrapezoidIntegrator T315 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
double z315 = T315.integrate(1000, F_ZERO, -1e-6, 1e-6);
System.out.printf("Z315 result=%.17g%n", z315);
// Z316 -----
headline("Z316 success (both flags surrogate; within budget)");
TrapezoidIntegrator T316 = new TrapezoidIntegrator(0.0, 0.5, 1,
2);
double z316 = T316.integrate(10, F_ZERO, -1.0, 1.0);
System.out.printf("Z316 result=%.17g%n", z316);
System.out.println("\n[Done] Z301-Z316 executed.");
}
}
class TItest {
public static void main(String[] args) {
// VT
TIIVT t1 = new TIIVT();
t1.Vtest();
// FT
TIIFT t2 = new TIIFT();
// t2.Ftest();
// Z3
TIZ3 t3 = new TIZ3();
// t3.Z3test();
}
}

```

- Coverage report

1. Test case number: 16
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ TrapezoidIntegrator.java	 80.0 %	16	4	20
> TIIVT	 100.0 %	2	0	2
▼ TrapezoidIntegrator	 100.0 %	14	0	14
TrapezoidIntegrator(double, double,	 100.0 %	2	0	2
TrapezoidIntegrator(int, int)	 100.0 %	2	0	2
doIntegrate()	 100.0 %	6	0	6
stage(BaseAbstractUnivariateIntegra	 100.0 %	4	0	4
> TItest		0	0	0

3. V-Method testing instruction coverage: 97.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ TrapezoidIntegrator.java	32.5 %	495	1,030	1,525
> Tltest	22.2 %	4	14	18
> TIVT	71.9 %	314	123	437
▼ TrapezoidIntegrator	97.3 %	177	5	182
TrapezoidIntegrator(double, double,	100.0 %	19	0	19
TrapezoidIntegrator(int, int)	100.0 %	17	0	17
doIntegrate()	100.0 %	54	0	54
stage(BaseAbstractUnivariateIntegra	100.0 %	87	0	87

4. Fuzz testing branch coverage: 71.4%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ TrapezoidIntegrator.java	60.0 %	12	8	20
> TIVT	0.0 %	0	2	2
▼ TrapezoidIntegrator	71.4 %	10	4	14
TrapezoidIntegrator(double, double,	50.0 %	1	1	2
doIntegrate()	83.3 %	5	1	6
stage(BaseAbstractUnivariateIntegra	100.0 %	4	0	4
> TIFT	100.0 %	2	0	2
> Tltest		0	0	0

5. Fuzz testing instruction coverage: 83%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ TrapezoidIntegrator.java	25.8 %	394	1,131	1,525
> TIVT	2.7 %	12	425	437
> Tltest	44.4 %	8	10	18
> TIFT	49.7 %	223	226	449
▼ TrapezoidIntegrator	83.0 %	151	31	182
TrapezoidIntegrator(double, double,	52.6 %	10	9	19
doIntegrate()	100.0 %	54	0	54
stage(BaseAbstractUnivariateIntegra	100.0 %	87	0	87

6. Z3 branch coverage: 71.4%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ TrapezoidIntegrator.java	60.0 %	12	8	20
> TIFT	0.0 %	0	2	2
> TIVT	0.0 %	0	2	2
▼ TrapezoidIntegrator	71.4 %	10	4	14
doIntegrate()	66.7 %	4	2	6
TrapezoidIntegrator(double, double,	100.0 %	2	0	2
stage(BaseAbstractUnivariateIntegra	100.0 %	4	0	4
> TIZ3	100.0 %	2	0	2
> Tltest		0	0	0

7. Z3 instruction coverage: 85.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ TrapezoidIntegrator.java	34.8 %	530	995	1,525
> TIFT	2.7 %	12	437	449
> TIVT	2.7 %	12	425	437
> TIZ3	76.3 %	335	104	439
> Tltest	83.3 %	15	3	18
▼ TrapezoidIntegrator	85.7 %	156	26	182
doIntegrate()	92.6 %	50	4	54
TrapezoidIntegrator(double, double,	100.0 %	19	0	19
stage(BaseAbstractUnivariateIntegra	100.0 %	87	0	87